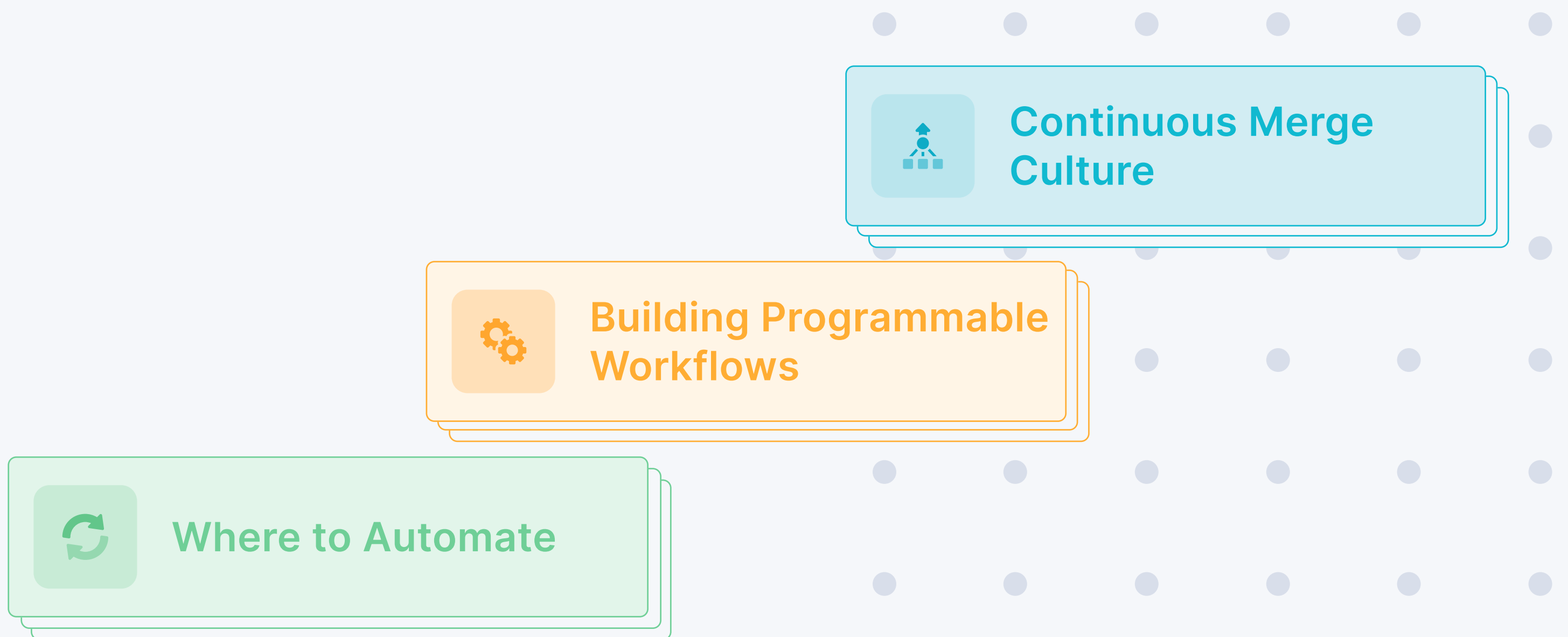




# Engineering Leader's Guide to Programmable Workflows

---

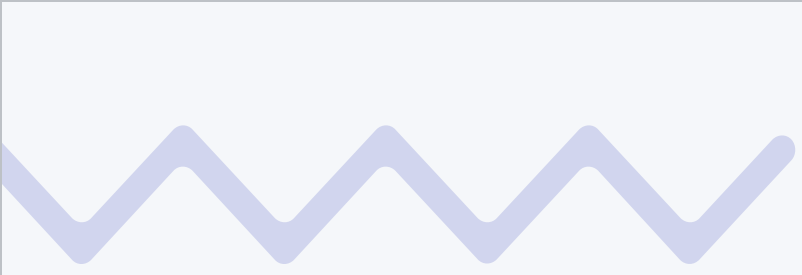
Metrics visibility, R&D spend awareness, performance benchmarks, and data-backed goals are all crucial for engineering leaders as they work to deliver operational excellence and drive business outcomes. However, all the insight in the world doesn't mean much without action. It's only through the targeted application of workflow automation in the development pipeline that gains in efficiency, quality, and the overall developer experience (DevEx) begin to materialize.



In this guide—part four of the five-part engineering success model—you’ll learn how to strategically apply automation that helps teams smooth out core development processes, reduce cognitive load, avoid context switching, and deliver higher quality code more efficiently.

Here’s what we’ll cover in the pages ahead:

- **Where to Automate:** Ensuring you’re applying automation to where developers are spending their time
- **Building Programmable Workflows:** Tactical guidance on what kind of automation to apply and the kind of outcomes you can expect to see as a result
- **Continuous Merge Culture:** Establishing and operationalizing a set of “merge standards” that remove friction in the pipeline and greatly simplify the biggest bottleneck in the development lifecycle: the PR process.



# Table of Contents

## **1. Automation–Not Metrics–Improve Engineering Teams**

“But I’ve Got CI/CD Tools That Automate My Dev Workflows.”

## **5. Applying Automation Strategically**

Communication tools

Git Repos

## **7. Results of Workflow Automation and How to Build It**

Operational Efficiency

Higher Quality

Merge Standard Enforcement at Scale

## **15. Conclusion**

# Automation–Not Metrics– Improve Engineering Teams

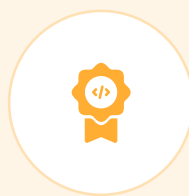
This may be a little bit blasphemous—especially for us who **wrote the guide on building a metrics program**—but metrics are just numbers. On their own, they don't do much. The uncomfortable truth about metrics is that without a way to act on the insight they provide, the only change they'll drive is putting another meeting on your already over-scheduled calendar. And nobody wants that.

That said, when engineering leaders apply automation against insights from metrics (**like DORA**), **performance benchmarks**, and **business alignment data**, they are able to deliver on their dual mandate of (1) operational excellence and (2) aligning engineering resources to business goals. In addition to these outcomes, automation removes friction from the developer pipeline—reducing toil, increasing merge frequency, and ensuring a great DevEx.

In short, automation is key to:



Increased Efficiency



Higher Quality



Better DevEx

## Increased Efficiency

Implementing automation across your entire SDLC streamlines the release process and enables developers to focus on what they really love – coding – instead of spending their time on manual merge tasks, data entry, and low value code reviews. And don't forget that because every company is a software company, the ability to merge code faster and at a higher quality enables your organization to gain a competitive advantage and deliver more value more quickly.



**WorkerB** APP 1:55 AM

Hey [@Betsy](#), skip the trip to Jira

**Create a ticket for this pull request?**

[PR #2395](#) linenv/Fix for BB Server username clone url

Create ticket

We suggest this because there is no ticket linked to this PR

[Learn more](#) | [How do I link a PR to a Jira ticket?](#)

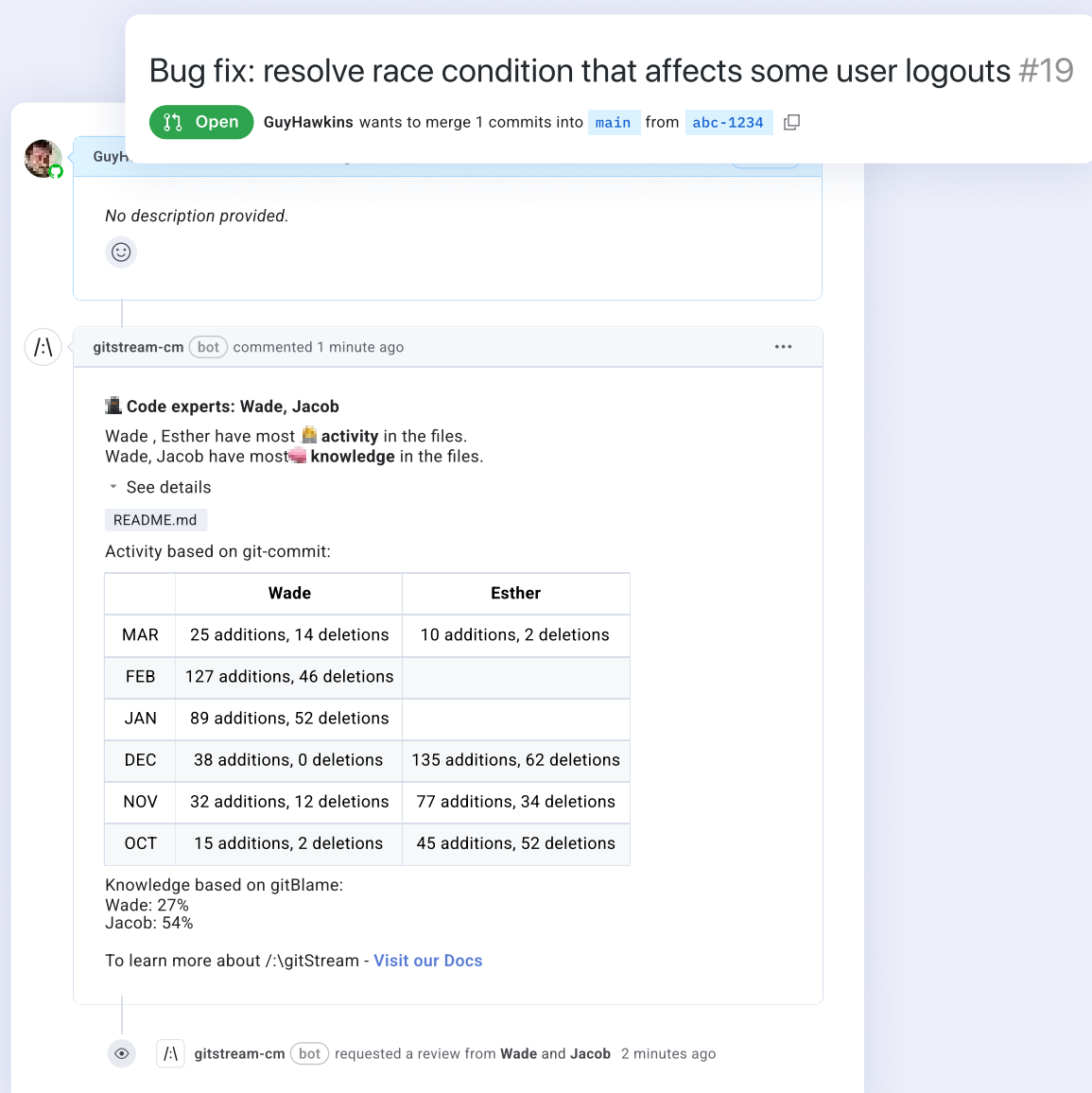
**A one-click workflow to add a Jira ticket to a PR**

# Automation–Not Metrics– Improve Engineering Teams

## Higher Quality

Automation and a culture of Continuous Merge facilitates more thorough reviews, leading to higher quality code, adherence to compliance best practices, and fewer bugs—all of which help drive increased engineering ROI and more value for the business.

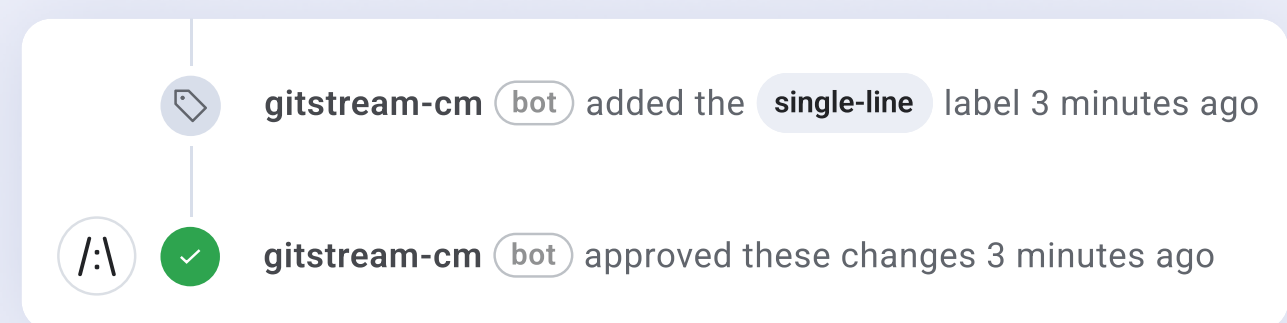
Additionally, automation that alerts developers to high-risk changes, like PRs where an expert reviewer is needed and PRs missing data objects, can help them identify and address problems early, ensuring higher stability over time. It's this standardization of merge processes that drive best practices across your organization.



## Code Experts automatically assigns PR reviews to the most qualified person on the team

# Better DevEx

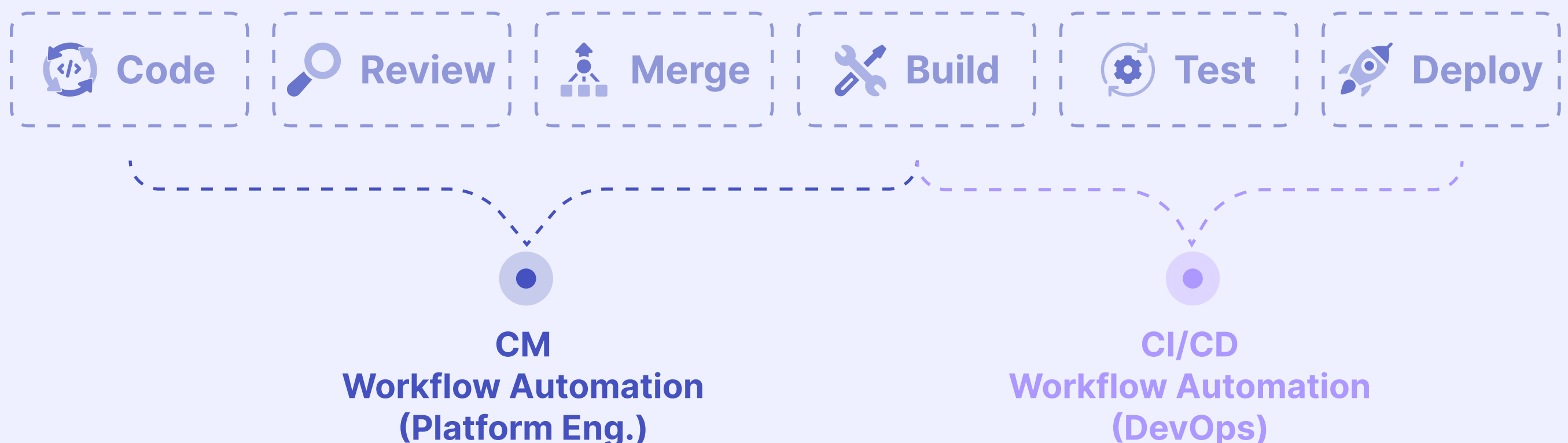
Automation that removes as much friction as possible in a developer's workflow reduces toil and enables them to merge higher quality code, more quickly. And workflow automation can help you expand codebase knowledge between teams and fast track ramp time for new hires. It also promotes a culture of continuous improvement and learning. This ultimately drives increased satisfaction, fosters good culture **(which is arguably more important than technology)**, and will help with talent retention (a key consideration for maintaining control over engineering spend).



## Automatically approve and merge low risk PRs that are labeled with "Safe Changes"

# Automation–Not Metrics– Improve Engineering Teams

CM and CI/CD complement each other and  
work together to streamline and improve the SDLC



## “But I’ve Got CI/CD Tools That Automate My Dev Workflows.”

The kind of workflow automation (and benefits) outlined above is called **Continuous Merge**—a concept we’ll explore in detail in a bit—but for now, just know that it’s what completes the promise of CI/CD. Ten years ago, CI/CD revolutionized software development and delivered amazing results like:

- More quality/predictability at higher speed and scale
- Faster time to market
- Better collaboration
- Reduced risk

And that’s just to name a few. Now everyone is adopting it in some way as it is a huge advantage...**BUT**

CI/CD doesn’t solve for merge frequency because the path to merging code back into the mainline exists in the murky area between coding and building (known as the “PR process” and “the most inefficient part of the SDLC”). A lot of developers' time is spent reviewing PRs—and even more specifically, reorienting themselves with code because so much time has elapsed since the PR was authored.

# Automation–Not Metrics– Improve Engineering Teams

## Did you know?

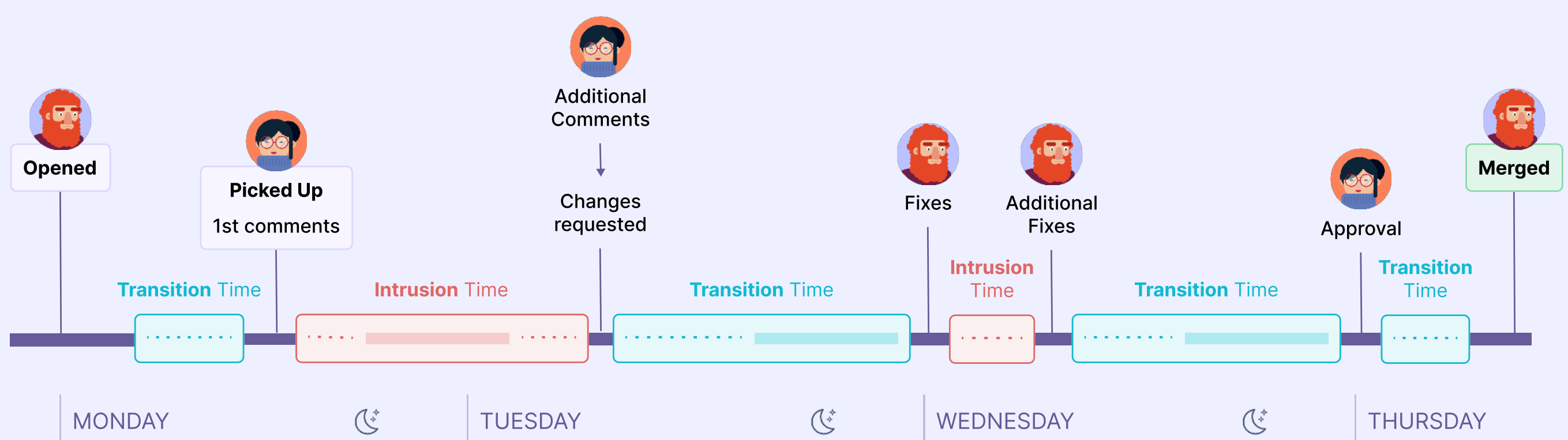
The average cycle time is 7 days and that idle time is 5 of those days?

That leads to many of those PRs getting a rubber stamp “LGTM” review in that 7-day period—possibly impacting quality and creating further delays. Besides work backlogs, delayed reviews can trigger complex merge conflicts. Even worse, pull requests come before integrated testing in the CI/CD pipeline, so a successful review doesn’t guarantee a similar outcome later. And every delay that results in days or weeks-long review cycles have significant cost implications.

Speeding up this process, shifting it left with Continuous Merge, and reducing idle time is how toil, cognitive load, escaped defects, and context switching is reduced—and it will have a similar effect and the same added benefit as CI/CD.

This guide will show you how to apply automation strategically in the right environments to see the results mentioned above and enable engineering leaders to build a Continuous Merge mindset in their organization and deliver on their dual mandate.

## The standard PR process has a lot of idle time and inefficiency baked in.



# Applying Automation Strategically

Before diving into the nitty gritty of programmable pipelines, let's quickly check in on where you are in your engineering improvement journey. If you're considering implementing developer pipeline and workflow automation you've likely:

- **Set up a robust engineering metrics program and identified your bottlenecks**
- **Incorporated crucial resource allocation and business alignment visibility**
- **Come up with your strategy for setting and reporting progress against improvement goals**

If you haven't read through our engineering leader's guides for these initiatives, start there by clicking the above links. Now back to automation.

## Where to Apply Workflow Automation

The first rule for building automated developer workflows: be very targeted and apply automation to where developers are spending their time. However automation can—and should—be applied cross-locationally (e.g. automation to add more context on a PR can be applied in git and in Slack).

Developers spend a lot of their time in meetings, in communication/collaboration tools like Slack and Teams, or in git repositories reviewing code. Let's look at why automation should be applied in these environments.

## Communication Tools

A lot of a dev's time is devoted to collaborating with peers, requesting code reviews, gathering additional requirements and clarification, etc. In the world of remote-first (quickly becoming a standard), communication and collaboration tools are right up there with an internet connection in terms of importance. And so to help developers adopt better habits (like keeping PRs small and jumping on reviews faster), reduce context switches, and generally improve quality and efficiency, it makes sense to apply automation here in the form of real-time updates on the work they're doing.



**LinearB Notifier** APP 11:49 AM

Hey @Jane

Miki requested your review for this pull request

[Migrator/LINB-2255](#) - [Open notifications center to everyone](#)

🕒 Est. review time **3 minutes** [how is this calculated?](#)

Type [/lb reviews](#) to see all of your reviews with estimation times

Here's an automated alert where one dev is asking another for a review. Notice the context of estimated review time, ticket number, repo, and who requested the review.

# Applying Automation Strategically

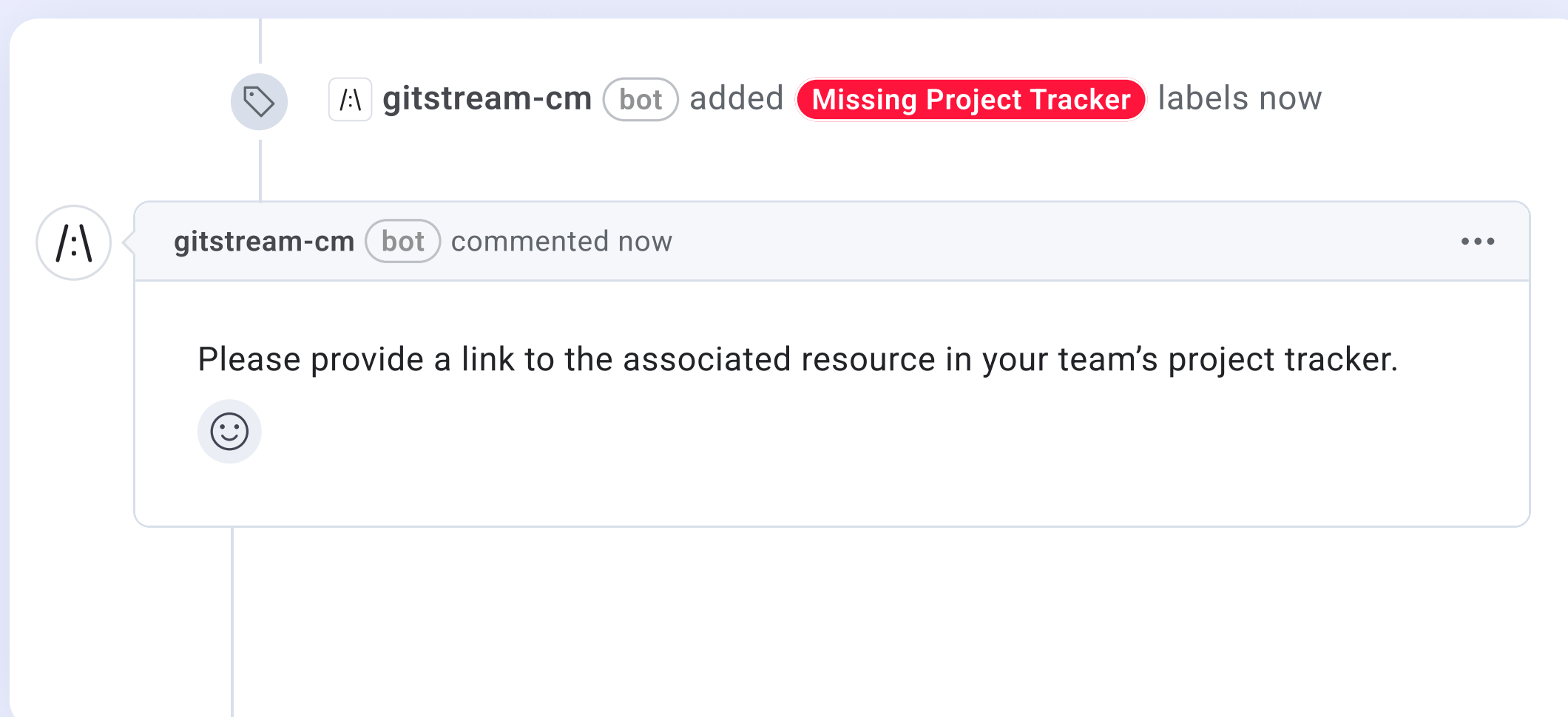
## Git Repos

Applying automation here changes the DNA of the PR process—more specifically, it ceases being “one size fits all” where every PR goes through the same gates, reviews, and approvals. Automation in this environment is the most direct path to a Continuous Merge mindset (more on this later). By applying automation, PR-policy-as-code, and enforcing merge standards for repos or the entire organization you’re optimizing and streamlining the PR process before a PR is even issued—making your teams more efficient and able to deliver with higher quality.

Generally speaking, the application of workflow automation in these areas is used to drive the following results:

-  Operational **Efficiency**
-  Improved **Quality of Work**
-  Merge Standard **Enforcement at Scale**

**This PR is missing a Jira ticket. Before any reviews take place, this oversight should be addressed to maintain SOC2 Compliance.**



# Results of Workflow Automation and How to Build It

## Operational Efficiency

This is all about offloading manual work, providing more context up front to smooth out processes, routing PRs more intelligently, and reducing developer toil—all of which gets devs back to what they love doing: coding.

Applying these automations—will help you improve your efficiency metrics (**think DORA metrics like cycle time and deployment frequency**) and net measurable results such as:

- Getting PRs in front of the right people immediately
- Streamlined collaboration for an average 44% decrease in cycle time
- Higher merge and deploy frequency—helping you deliver more features and improve your DevEx

These efficiency gains are enabled through:



Pull request labeling



Real time alerting in communication tools



Workflow automation that speeds up the merge process and fine tunes your CI engine

# Results of Workflow Automation and How to Build It

## 1. Pull Request Labeling and Context

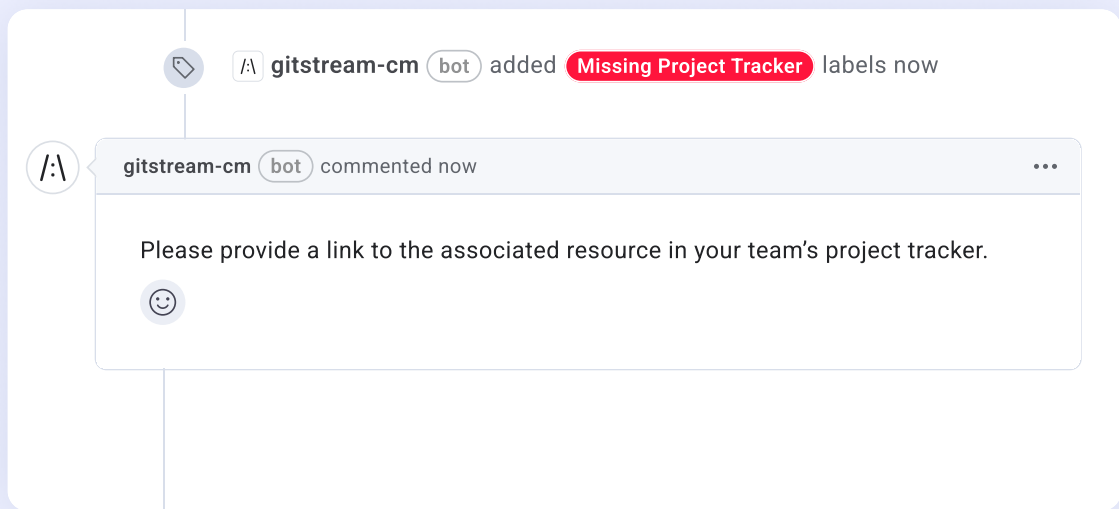
Context and visibility are also key to streamlining the code delivery process. A famous shrimping magnate and early investor in Apple once said “Life PRs are like a box of chocolates.” Ok fine he didn’t say that. But it’s kind of the same thing: you really don’t know what you’ve got until you bite into it.

It could be a small, well-organized PR that focuses on a single problem, uses correct components, has all docs and requirements in place, and takes minutes to review—the PR equivalent of a delicious caramel. It could be a PR with 500+ diffs, lots of cascading dependencies –

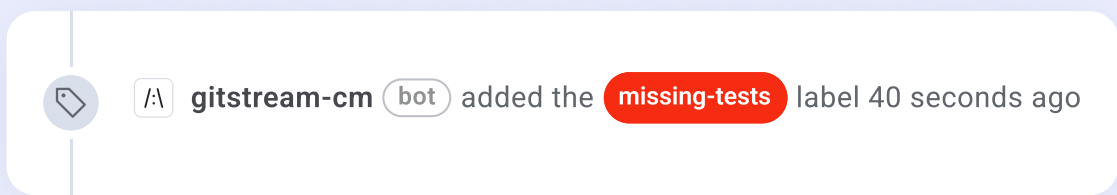
using deprecated APIs, and requiring carving time out of the day to review—the PR equivalent of gross coconut (no offense to anyone that likes the coconut ones).

One of the best confectionery innovations was a guide in the top of the box letting you know which chocolate was which, so you can pick the one you want to eat. The same approach should be applied to PRs in a culture of Continuous Merge. **With a simple automated .cm file**—which can be applied to individual repos or rolled out org wide—you can automatically label PRs with information that helps your reviewers avoid (but more realistically prepare themselves) for the “coconut filling PRs” like the one with 80% refactoring and flaky tests.

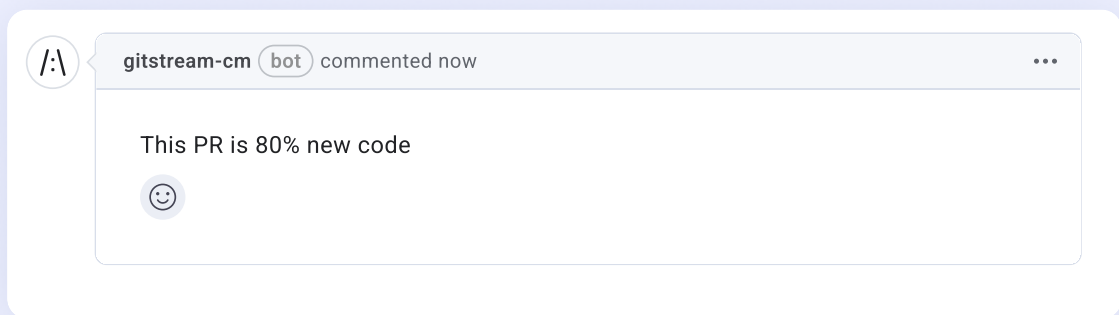
### PR labels provide more context to decrease review time and cognitive load



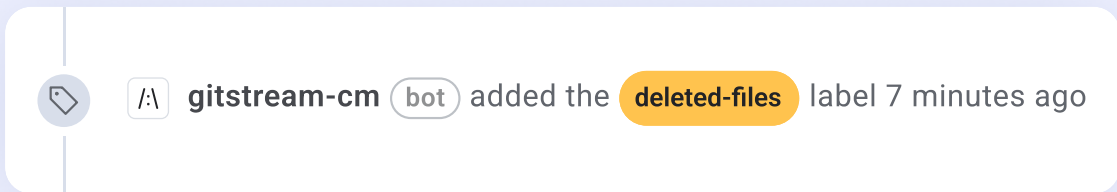
Missing a Jira ticket



Missing testing



This PR is 80% new code



This PR deleted files

# Results of Workflow Automation and How to Build It

## 2. Real Time Alerting in Communication Tools

Software development is a team sport and collaboration is key. Automation applied here keeps best practices top of mind, reduces idle time, maintains visibility into work, and ensures focus isn't split.

Below you'll find some automated alerts that help teams minimize manual work and context switching -- both of which are common speed bumps for dev teams.

Additional communication tool efficiency automations include:

- Enabling devs to **prioritize their daily task list and maintain visibility into what needs to get done.**
- **Sharing unassigned PRs** with a specified channel to speed up reviews and help the team move work forward.
- **Tagging people in git, which alerts them in Slack and Teams** to increase visibility and ensure nothing slips through the cracks.
- **Keeping teams aligned to PR size reduction goals.** When PRs are small, they're picked up, reviewed, and approved faster--all of which increase efficiency.

### Some example automated alerts

**WorkerB** APP 1:55 AM

Hey [@Betsy](#) , skip the trip to Jira  
**Create a ticket for this pull request?**  
PR #2395 linenv/Fix for BB Server username clone url  

Create ticket

We suggest this because there is no ticket linked to this PR  
[Learn more](#) | [How do I link a PR to a Jira ticket?](#)

30% of PRs are missing a Jira ticket and data entry is no fun. Automate the process with one-click workflows from Slack or MS Teams

**WorkerB** APP 8:33 AM

Hey [@Betsy](#) , skip the trip to Jira  
Miki requested your review for this pull request  
[linenv/WWF-418-linrest-sensors-api-fix](#)  
🕒 Est. review time: **up to 2 min** [Learn more](#)  

changes/changes.yml CHANGED

1

- api-gateway: v0.13.20

2

+ linrest: v0.22.28

3

+ pm-connectors: v0.18.48

☒ Add your approval comment

LGTM

Approve

Avoid context switching and review/approve small PRs directly from Slack

# Results of Workflow Automation and How to Build It

## 3.Git Workflow Automation

Here is where the rubber hits the road to build a highly efficient PR process. When thinking about the relationship between PR authors and reviewers a key consideration is Work in Progress (WIP). When a PR is issued, WIP doubles because it’s not just the original PR anymore, now the review is also WIP in somebody else’s queue.

With the “standard” PR process, this WIP count can quickly spiral out of control causing delivery delays, cognitive overload, and burnout.

If you can automate some of this in git-like auto-approving safe changes that only impact README.md files, for example—you can free up resources to take on more impactful, high-value projects and avoid WIP overload.

### Workflow automation in git to streamline merge and free up resources

gitstream-cm (bot) added the **safe-changes** label now

gitstream-cm (bot) approved these changes now

Auto-Approvals:

Fast track PR merges for safe changes (like docs), expert teams, and clean code

CI Orchestration:

Selectively trigger CI Pipelines

All checks have passed

1 action required and 5 successful checks

✓

Server CI PR/pr-ci/Check mocks (pull\_request)

Successful in 2m

Details

✓

Server CI PR/pr-ci/Check go mod tidy (pull\_request)

Successful in 57s

Details

✓

Server CI PR/pr-ci/golangci-lint (pull\_request)

Successful in 6m

Details

✓

Server CI PR/pr-ci/Check serialization methods for hot structs (pull\_request)

Successful in...

Details

✓

Server CI PR/pr-ci/Check style (pull\_request)

Successful in 3m

Details

✓

Server CI PR/pr-ci/Check migration files (pull\_request)

Successful in 9s

Details

✓

This branch has no conflicts with the base branch

Only those with write access to this repository can merge pull requests.

gitstream-cm (bot) requested a review from my-organization/mentors 1 minute ago

gitstream-cm (bot) added the **new-contributor** 1 minute ago

gitstream-cm (bot) left a comment

Welcome Newcomer:

Share knowledge across teams

# Results of Workflow Automation and How to Build It

## Higher Quality

This collection of workflow automation is all about: ensuring best practices are followed, reducing complexity (the enemy of efficiency and security), and mitigating risk—all of which drive higher code quality.

### Don't Forget...

Higher quality also impacts efficiency and the bottom line. If you're not having to rework new code or spend lots of time on code reviews, you'll ship more code and generate more value faster.

Applying these workflow automations will help you improve your quality and stability metrics. **In the world of DORA, this is CFR and MTTR.** When teams work to improve code quality, they tend to see measurable results such as:

- Fewer bugs
- Less churn
- More value generation and engineering ROI

Like Operational Efficiency workflows, quality improvements are enabled through real time alerting, pull request labels, and workflow automation that enforces best practices.

## 1. Real Time Alerting in Communication Tools

Just like efficiency workflows, you can apply automation to collaboration tools that will let developers know about work that may impact quality—helping them adopt good habits, stay aligned to improvement goals, and deliver better code.

### Real time alerts that improve quality



**WorkerB** APP 10:22 AM

A pull request has been merged containing 167 code changes with 82% refactor and rework

**Goal:** Avoid merging pull requests with work at risk

PR: [linweb/Revert "LINBEE-3503 | Remove pickers from Pulse CTA page"](#)

PR Owner: Oriel Zaken

[?](#) [More large PRs](#)

Avoid merging risky work (large PRs with high rework and refactor rates)



**WorkerB** APP 1:55 AM

A pull request with 168 code changes was merged without review

**Goal:** Review all PRs with more than 40 code changes

PR: [infra/Reloader](#)

[?](#) [More unreviewed PRs](#)

Avoid merging PRs that haven't been reviewed

# Results of Workflow Automation and How to Build It

Additional communication tool automations that drive quality include: **Setting and keeping the team aligned to PR size reduction goals.**

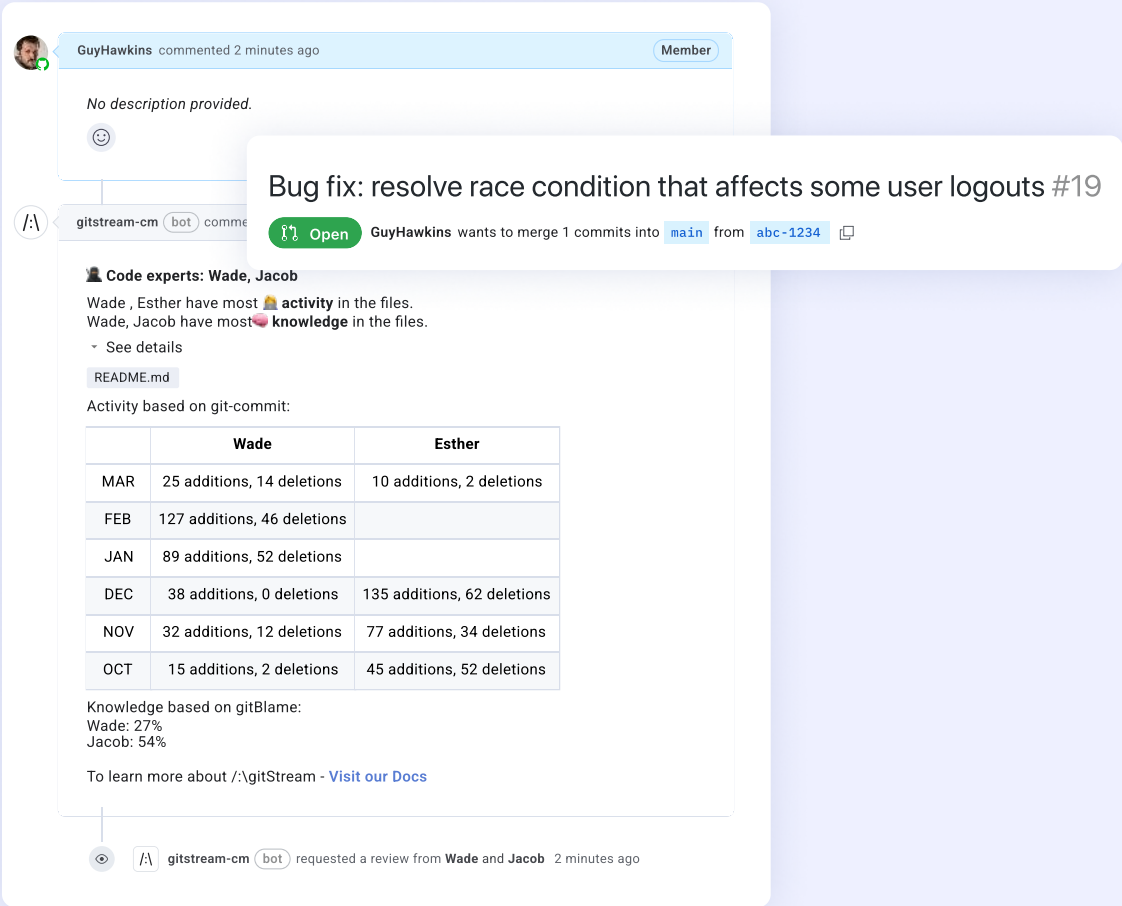
PR size is a key leading indicator of quality because less code means less of a chance to introduce bugs and break something in production which is what informs CFR. And if/when something does break, less code to review means finding and fixing the problem faster—driving MTTR down.

## 2. Pull Request Labeling and Workflow Automation in git

This collection of automations provide both workflows and labeling in a single operation. When applied to a repo, across multiple repositories, or the entire org, these workflows make it easier for teams to:

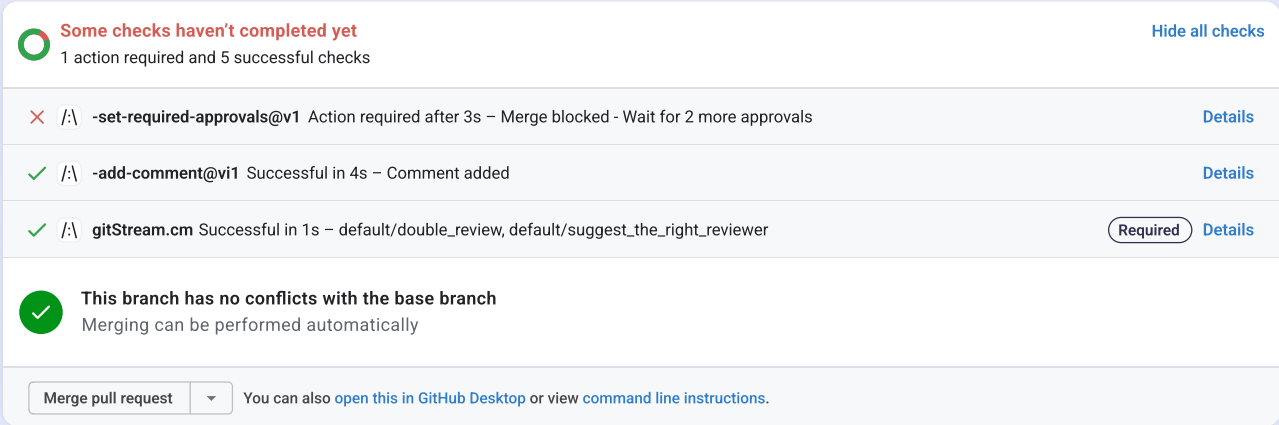
- Properly document and structure their code
- Prevent bugs before they happen
- Fix issues quickly

### Workflow Automation in git to Improve Quality



#### Code Experts:

Identify and automatically assign the best reviewer, with the most knowledgeable, who was recently active to review code—improving quality while decreasing developer toil



#### Review sensitive files:

Define a custom list of files that trigger additional reviews such as:

- PRs with 200+ code changes
- PRs opened by new team members
- PRs with high potential for security risks
- PRs that affect complex components in the codebase

# Results of Workflow Automation and How to Build It

## 7 Additional Workflow Automations That Stabilize Delivery:

-  **Missing Tests**  
Mark PRs without tests
-  **Deprecated API Alert**  
Generate PR label & trigger a change request to use preferred API
-  **High-Risk Tags**  
Let DevSecOps take the reins on high-risk work
-  **Flag Security Alerts**  
Get alerted when quality gate checks fail to pass
-  **Review Duplications**  
Flag PRs with more than 3% duplicated code.
-  **Auto Links to Project Tracking**  
Auto post comments linking to your issue tracker
-  **SonarCloud Quality Reports**  
Get more context into the security posture of code

Learn even more about stability metrics with automation in the [Continuous Merge Guide to DORA Metrics](#).

## Merge Standard Enforcement at Scale

A constant for dev teams is that they operate in many different ways. Most organizations have a set of coding standards and processes everyone follows—but very few (if any) are thinking about merge standards.

### Merge Standards

A set of guidelines that standardize when and how code changes should be merged into a shared codebase.

These are policies and goals an organization and team can set and enforce at scale (across repos or the entire organization) using policy-as-code. Merge standards dictate the decision for “if” “then” scenarios. When **building a culture of Continuous Merge** within your teams, your merge standards should balance velocity with quality.

Using tools like **WorkerB** and **gitStream**, teams can set and easily adhere to merge standards.

### Continuous Merge (CM)

The practice of automating the classification of pull requests in order to optimize the path to merge.

# Results of Workflow Automation and How to Build It

To start, we recommend working with your team to evaluate your existing code merge practices. It can be helpful to ask questions like:

- What's helping us ensure high code quality?
- When we collaborate well, what tend to be the reasons?
- What processes enable devs to get code out the door quicker?

Now that you've aligned with your team on what's most important to standardize, it's time to create a comprehensive document outlining your team's new merge standards. Feel free to pull from our team's guide below or create your own from scratch.

From there, publish your merge standards somewhere easily accessible, like on your team's–

wiki, then conduct a training session to talk through best practices and address any questions or concerns. It can also be helpful to record this session for future hires to review during the onboarding process.

Implementing this practice across repos–or the entire org–removes confusion and ambiguity with code merges, helping scaling organizations optimize processes and future proof their operations. Over time, this will reduce time spent delivering status updates, pinging colleagues for review requests, or toiling away in PM tools. Your engineers will ship more code, and have a better overall developer experience.

Moving forward, because these standards are subject – and even likely – to change, we recommend evaluating and updating your merge standards at least once a year.

## Merge Standards That Enforce Quality

1. Assign the correct reviewers.
2. Assign DevSecOps team to review high-risk code.
3. Enforce development best practice.
4. Link a JIRA ticket to every PR.
5. Require extra reviewers for complex or error-prone changes.
6. Ensure changes include required documentation.
7. Ensure that all dependencies have already been merged.
8. Unify your commits regularly using git rebase.

## Merge Standards That Boost Efficiency

1. Keep changes small when possible.
2. Limit complexity, focus on a single problem.
3. Automatically merge safe changes (e.g. documentation, tests, non-functional formatting).
4. Automate change requests for things like deprecated APIs.
5. Notify watch list for code changes.

# Conclusion

Congratulations, you've now provided a holistic automation framework for your development teams that will reduce toil, standardize best practices, and enable them to actively improve. Your engineering teams will now be able to deliver code more efficiently, with higher quality, and ensure those results at scale.

Look out for the next guide in this series where we'll explore how to tie everything together to ensure predictability in project delivery for your engineering organization.

# Conclusion

LinearB's software delivery management platform is ideal for building an engineering metrics program from scratch and aligning engineering to the needs of the business. After onboarding and initial configuration/integration with your existing software stack, you'll get the metrics, dashboards, insights, and automation discussed in this guide right out of the box.

You can **get your DORA metrics with a free forever account** and begin building your engineering metrics program today! When you're ready to incorporate workflow automation into your improvement initiative, all of the gitStream automations and .cm files discussed in this guide **are available for free**. If you'd like to discuss any of what was covered in this guide in more detail, or you want to see Programmable Workflows in action, you can **schedule a demo**.

We'd love to hear from you!